Buổi 9: Lập trình theo hướng phản ứng và khung hiển thị

Nội dung chính

9.1. Lập trình khai báo với Jetpack Compose.

9.2. Tạo giao diện kết hợp giữa Compose và Views truyền thống.

Mục tiêu cần đạt được

Hiểu cách lập trình khai báo và các thành phần cơ bản trong Jetpack Compose.

Biết cách sử dụng ViewModel và LiveData để quản lý trạng thái trong ứng dụng.

Vận dụng Jetpack Compose để xây dựng giao diện hiện đại và tối ưu hiệu năng.

Tích hợp giao diện Jetpack Compose vào ứng dụng sử dụng Views truyền thống.

9.1. Các phương pháp lập trình cho thiết bị di động

9.1.1. Tạo giao diện ứng dụng theo hướng phản ứng cho ứng dụng Android

Jetpack Compose là một toolkit UI hiện đại của Android, được thiết kế để xây dựng giao diện người dùng (UI) hiệu quả và nhanh chóng. Compose cung cấp một cách tiếp cận hoàn toàn mới cho việc phát triển UI trong Android, thay thế cách tiếp cận truyền thống với XML bằng lập trình khai báo. Với Compose, chỉ cần mô tả trạng thái của UI thay vì lập trình trực tiếp để thay đổi giao diện người dùng dựa trên các sự kiện.

9.1.2. Lợi ích của Jetpack Compose

- **Dễ đọc và bảo trì**: Giao diện được định nghĩa bằng cách mô tả trạng thái của nó, giúp code dễ đọc hơn, dễ bảo trì và ít lỗi.
- **Tái sử dụng**: Compose cho phép tái sử dụng các thành phần UI dễ dàng hơn nhờ vào tính chất module.
- **Tích hợp tốt với Kotlin**: Compose được viết hoàn toàn bằng Kotlin, giúp tận dụng tối đa ngôn ngữ này để viết code ngắn gọn, an toàn và dễ hiểu.
- **Hỗ trợ theme động**: Compose hỗ trợ hệ thống theme động và dễ dàng tạo các chủ đề tùy chỉnh cho ứng dụng.
- **Tương thích với Views**: Compose có thể hoạt động song song với các thành phần UI truyền thống trong Android, giúp dễ dàng chuyển dần từ Views sang Compose.

9.1.3. Cài đặt Jetpack Compose

Để sử dụng Jetpack Compose trong dự án Android, cần thêm các thư viện cần thiết vào file build.gradle của module:

```
android {
    // Thiết lập Compose
    buildFeatures {
        compose true
    }
    composeOptions {
        kotlinCompilerExtensionVersion '1.5.0'
    }
}
```

```
dependencies {
    implementation "androidx.compose.ui:ui:1.0.0"
    implementation "androidx.compose.material:material:1.0.0"
    implementation "androidx.compose.ui:ui-tooling:1.0.0"
    implementation "androidx.lifecycle:lifecycle-runtime-ktx:2.3.1"
    implementation "androidx.activity:activity-compose:1.3.0"
}
```

9.1.4. Thành phần cơ bản của Compose

Compose được xây dựng dựa trên các Composable functions. Đây là các hàm đặc biệt giúp mô tả các phần tử UI.

Tạo Composable function

Mỗi thành phần UI trong Compose được định nghĩa bằng một Composable function, được đánh dấu bằng annotation @Composable. Ví dụ:

```
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}
```

Hàm Greeting trên là một Composable function đơn giản hiển thị dòng chữ "Hello" kèm theo tên được truyền vào.

Gọi Composable function

Để gọi một Composable function, chỉ cần gọi nó trực tiếp từ bên trong một hàm Composable khác. Compose sử dụng một công cụ gọi là "Compose Compiler" để quản lý việc vẽ lại UI khi trạng thái thay đổi.

```
@Composable
fun MyApp() {
    Greeting(name = "Android")
}
```

Sử dụng trong Activity

Trong một ứng dụng Android sử dụng Compose, thay vì thiết lập giao diện thông qua setContentView(), sử dụng hàm setContent để đặt Composable vào màn hình:

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MyApp()
        }
    }
}
```

Các thành phần UI cơ bản

Text

Thành phần Text là một Composable function cơ bản để hiển thị văn bản:

```
@Composable
```

```
fun SimpleText() {
    Text("Hello Compose!")
}
```

Button

Thành phần Button cho phép người dùng tương tác với ứng dụng. Kết hợp với onClick để xử lý sự kiện nhấp chuột:

```
import androidx.compose.material.Button
@Composable
fun SimpleButton() {
    Button(onClick = { /* Thực hiện một hành động */ }) {
        Text("Nhấn tôi")
    }
}
```

Box, Row và Column

Trong Jetpack Compose, Box, Row và Column là ba thành phần bố cục (layout) cơ bản dùng để sắp xếp và căn chỉnh các thành phần giao diện trong ứng dụng Android. Chúng cung cấp cách tiếp cận linh hoạt để quản lý giao diện người dùng mà không cần sử dụng XML như trong phương pháp truyền thống.

- **Box**

Trong Jetpack Compose, Box là một thành phần bố cục (layout) dùng để xếp chồng các thành phần con lên nhau, tương tự như FrameLayout trong Android truyền thống. Box rất hữu ích khi cần chồng các thành phần hoặc căn chỉnh chúng theo vị trí tương đối trong không gian của Box. Cú pháp tạo ra một Box:

```
Box {
    Text("Text on top")
    Button(onClick = { /* Do something */ }) {
        Text("Button below")
    }
}
```

Trong ví dụ này, các thành phần Text và Button được xếp chồng lên nhau theo thứ tự xuất hiện trong mã.

- **Row**

Trong Jetpack Compose, Row là một thành phần bố cục (layout) được sử dụng để sắp xếp các phần tử con theo chiều ngang (từ trái sang phải). Row rất linh hoạt, cho phép đặt các thành phần con cạnh nhau trong cùng một hàng và có thể tùy chỉnh vị trí của chúng bằng cách sử dụng các thuộc tính như horizontalArrangement (sắp xếp các phần tử theo chiều ngang) và verticalAlignment (căn chỉnh theo chiều dọc).

Cú pháp cơ bản để tạo một Row như sau:

```
Row {
    Text("Item 1")
    Text("Item 2")
    Text("Item 3")
}
```

Trong ví dụ này, các thành phần Text sẽ được sắp xếp theo hàng ngang. Mặc định, các thành phần này sẽ được sắp xếp liền kề nhau mà không có khoảng cách.

Column

Thành phần Column trong Jetpack Compose là một layout container được sử dụng để sắp xếp các phần tử con theo chiều dọc. Các phần tử bên trong Column được đặt lần lượt từ trên xuống dưới theo thứ tự khai báo. Có thể tùy chỉnh vị trí các thành phần bằng cách sử dụng các thuộc tính như `verticalArrangement` (sắp xếp các phần tử theo chiều dọc) và `horizontalAlignment` (căn chỉnh theo chiều ngang)

Cú pháp cơ bản:

```
Column(
    verticalArrangement: Arrangement = Arrangement.Top,
    horizontalAlignment: Alignment = Alignment.Start,
    content: @Composable ColumnScope.() -> Unit
)
```

Modifier

Modifier trong Compose được sử dụng để thay đổi, định dạng và thêm hành vi cho các Composable. Modifier giúp điều chỉnh kích thước, căn chỉnh, lề, padding, và nhiều thuộc tính khác.

Ví dụ, để thêm padding cho một thành phần:

```
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
@Composable
fun PaddedText() {
    Text(
        text = "Compose với padding",
        modifier = Modifier.padding(16.dp)
    )
}
```

- **Sử dụng nhiều Modifier**

Có thể xâu chuỗi nhiều Modifier lại với nhau bằng cách sử dụng toán tử .. Ví dụ, muốn thêm padding, border, và thay đổi kích thước cho một Box:

```
Box(
    modifier = Modifier
        .padding(16.dp)
        .border(2.dp, Color.Red)
        .size(100.dp)
)
```

Ở đây, Modifier sẽ:

- Thêm padding 16dp xung quanh Box.
- Thêm một border màu đỏ với độ dày 2dp.
- Đặt kích thước của Box là 100dp x 100dp.
- **Modifier cho tương tác**

Modifier không chỉ dùng cho bố cục, mà còn dùng để xử lý tương tác, chẳng hạn như click. Ví dụ, có thể sử dụng Modifier.clickable để thêm hành động khi người dùng nhấn vào một thành phần:

```
Text(
    text = "Click me",
    modifier = Modifier
        .padding(16.dp)
        .clickable {
            // Xử lý sự kiện click
        }
)
```

- **Căn chỉnh với Modifier**

Có thể sử dụng Modifier để căn chỉnh các thành phần UI bên trong các container như Box, Row, Column.

- **Căn chỉnh các thành phần trong Box**

Có thể căn chỉnh các thành phần con trong Box bằng cách sử dụng thuộc tính contentAlignment. Ví dụ, để căn chỉnh một Text ở giữa Box:

```
Box(
    modifier = Modifier.fillMaxSize(),
    contentAlignment = Alignment.Center
) {
    Text("Centered Text")
}
```

Ở đây, Text sẽ được căn giữa cả theo chiều ngang và chiều dọc trong Box.

Nếu muốn căn chỉnh từng thành phần riêng lẻ, có thể sử dụng Modifier.align(). Ví dụ, một thành phần nằm góc trên bên trái và một thành phần nằm góc dưới bên phải:

```
Box(
    modifier = Modifier.fillMaxSize()
) {
    Text(
        text = "Top Left",
        modifier = Modifier.align(Alignment.TopStart)
    )
    Text(
        text = "Bottom Right",
        modifier = Modifier.align(Alignment.BottomEnd)
    )
}
```

Thêm nhiều Modifier vào Box

Box cũng hỗ trợ các Modifier để điều chỉnh kích thước, thêm border, hoặc padding. Ví dụ, thêm một Box có kích thước cố định và border:

```
Box(
    modifier = Modifier
        .size(200.dp)
        .border(2.dp, Color.Red)
) {
    Text("Text inside box")
}
```

Trong ví dụ này, Box sẽ có kích thước 200dp x 200dp và viền đỏ.

Chồng các thành phần lên nhau

Box rất hữu ích khi muốn chồng các thành phần lên nhau. Ví dụ, hiển thị một ảnh với văn bản chồng lên:

```
Box {
    Image(
        painter = painterResource(id = R.drawable.image),
        contentDescription = null
    )
    Text(
        text = "Text on Image",
        modifier = Modifier.align(Alignment.Center),
        color = Color.White
    )
}
```

Trong ví dụ này, Text được chồng lên ảnh và căn giữa.

Sử dụng Modifier với Row

Row hỗ trợ các Modifier giống như các thành phần UI khác trong Compose. Có thể sử dụng các Modifier để điều chỉnh kích thước, căn chỉnh và thêm tương tác. Ví dụ, để tạo một Row chiếm toàn bộ chiều rộng của màn hình:

```
Row(
    modifier = Modifier.fillMaxWidth()
) {
    Text("Item 1")
    Text("Item 2")
    Text("Item 3")
}
```

Căn chỉnh các thành phần trong Row

Căn theo chiều dọc (verticalAlignment): Sử dụng thuộc tính này để điều chỉnh vị trí của các thành phần con theo chiều dọc trong Row. Ví dụ:

```
Row(
    verticalAlignment = Alignment.CenterVertically
) {
    Text("Item 1")
    Text("Item 2")
}
```

Trong ví dụ này, các thành phần sẽ được căn giữa theo chiều dọc trong Row.

Sắp xếp theo chiều ngang (horizontalArrangement): Dùng để kiểm soát cách sắp xếp các thành phần con theo chiều ngang. Ví dụ:

```
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
) {
    Text("Item 1")
    Text("Item 2")
    Text("Item 3")
}
```

Trong ví dụ trên, các thành phần con sẽ được sắp xếp với khoảng cách đều nhau ở hai đầu và giữa các thành phần.

Sử dụng weight trong Row

Có thể dùng Modifier.weight() để chia đều không gian giữa các thành phần con trong Row. Ví dụ:

```
Row(
    modifier = Modifier.fillMaxWidth()
) {
    Text(
        text = "Item 1",
        modifier = Modifier.weight(1f)
    )
    Text(
        text = "Item 2",
        modifier = Modifier.weight(1f)
    )
    Text(
        text = "Item 3",
        modifier = Modifier.weight(1f)
    )
}
```

Ở đây, mỗi Text sẽ chiếm một phần bằng nhau trong Row.

6. Thêm Modifier cho từng thành phần trong Row

Có thể áp dụng các Modifier riêng lẻ cho từng thành phần bên trong Row. Ví dụ, nếu muốn thêm border cho từng thành phần:

```
Row {
    Text(
        text = "Item 1",
        modifier = Modifier
            .border(1.dp, Color.Red)
            .padding(8.dp)
    )
    Text(
        text = "Item 2",
        modifier = Modifier
            .border(1.dp, Color.Blue)
            .padding(8.dp)
    )
    Text(
        text = "Item 3",
        modifier = Modifier
            .border(1.dp, Color.Green)
            .padding(8.dp)
    )
}
```

7. **Ví dụ tổng hợp**

Dưới đây là một ví dụ tổng hợp, kết hợp nhiều thuộc tính của Row:

```
Row(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp),
    horizontalArrangement = Arrangement.SpaceAround,
    verticalAlignment = Alignment.CenterVertically
```

```
) {
    Text("Item 1")
    Text("Item 2")
    Button(onClick = { /* Do something */ }) {
        Text("Button")
    }
}
```

Trong ví dụ này:

- Row chiếm toàn bộ chiều rộng của màn hình và có khoảng cách padding 16dp.
- Các thành phần được sắp xếp cách đều nhau nhờ Arrangement.SpaceAround.

Các thành phần được căn giữa theo chiều dọc.

Với column

Để căn chỉnh thành phần Column trong Jetpack Compose bằng cách sử dụng Modifier, có thể kết hợp các modifier như fillMaxSize(), padding(), background(), và align() để điều chỉnh kích thước, vị trí và các đặc điểm khác của Column.

Một số modifier phổ biến khi căn chỉnh Column:

- fillMaxSize(): Làm cho Column chiếm toàn bộ không gian có sẵn của màn hình.
- padding(): Thêm khoảng cách xung quanh Column để tránh đụng các phần khác.
- align(): Dùng để căn chỉnh Column trong container lớn hơn (ví dụ, một Box).

Ví dụ chi tiết:

1. Căn giữa Column trong toàn màn hình:

```
Column(
    modifier = Modifier
        .fillMaxSize() // Chiếm toàn bộ màn hình
        .padding(16.dp) // Thêm khoảng cách 16dp ở mọi phía
        .background(Color.LightGray), // Thêm nền màu xám nhạt
    verticalArrangement = Arrangement.Center, // Căn giữa theo chiều
dọc
    horizontalAlignment = Alignment.CenterHorizontally // Căn giữa
theo chiều ngang
) {
    Text(text = "Item 1")
    Text(text = "Item 2")
}
```

Trong ví dụ này:

- fillMaxSize() giúp Column chiếm toàn bộ kích thước màn hình.
- padding(16.dp) thêm khoảng trống 16dp xung quanh Column.
- background(Color.LightGray) thêm màu nền cho Column.
- Arrangement.Center và Alignment.CenterHorizontally giúp căn giữa nội dung bên trong Column.

2. Căn Column ở góc dưới bên phải màn hình:

```
Box(
    modifier = Modifier.fillMaxSize() // Chiếm toàn bộ kích thước màn
hình
) {
    Column(
        modifier = Modifier
            .align(Alignment.BottomEnd) // Căn `Column` ở góc dưới bên
phải
            .padding(16.dp), // Thêm padding 16dp từ cạnh màn hình
        verticalArrangement = Arrangement.spacedBy(8.dp) // Thêm
khoảng cách giữa các phần tử
    ) {
        Text(text = "Item 1")
        Text(text = "Item 2")
    }
}
```

Trong ví dụ này:

- Box bao bọc Column và làm cho nó chiếm toàn bộ màn hình.
- align(Alignment.BottomEnd) giúp căn Column vào góc dưới bên phải của màn hình.
- padding(16.dp) đảm bảo có khoảng trống 16dp giữa Column và các cạnh màn hình.

3. Căn Column dọc theo cạnh trái màn hình:

```
Column(
    modifier = Modifier
        .fillMaxHeight() // Chiều cao của `Column` lấp đầy toàn bộ màn
hình
        .width(100.dp) // Đặt chiều rộng cố định là 100dp
        .background(Color.Cyan) // Màu nền xanh da trời
        .padding(8.dp), // Padding 8dp từ cạnh màn hình
    verticalArrangement = Arrangement.SpaceBetween // Căn đều khoảng
cách giữa các phần tử
) {
    Text(text = "Item 1")
    Text(text = "Item 2")
    Text(text = "Item 3")
}
```

Trong ví dụ này:

- fillMaxHeight() làm cho Column chiếm toàn bộ chiều cao của màn hình.
- width(100.dp) giới hạn chiều rộng của Column là 100dp.
- Arrangement.SpaceBetween chia đều khoảng cách giữa các phần tử trong Column.

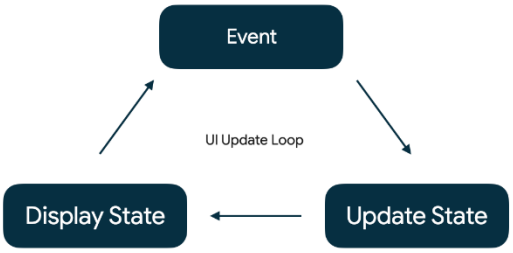
9.1.5. Quản lý trạng thái trong Compose

State của ứng dụng là giá trị bất kỳ có thể thay đổi theo thời gian (ví dụ: tin nhắn cuối cùng nhận được trong ứng dụng nhắn tin). Nhưng điều gì giúp cập nhật trạng thái? Trong các ứng dụng Android, trạng thái được cập nhật để phản hồi các sự kiện.

Sự kiện là các thông tin đầu vào được tạo từ bên ngoài hoặc bên trong một ứng dụng, ví dụ như:

- Người dùng tương tác với giao diện người dùng bằng cách nhấn một nút.
- Các yếu tố khác, chẳng hạn như cảm biến gửi giá trị mới hoặc phản hồi mạng.

Mặc dù trạng thái của ứng dụng cung cấp mô tả về nội dung sẽ hiển thị trong giao diện người dùng, nhưng sự kiện là cơ chế thay đổi của trạng thái, dẫn đến các thay đổi đối với giao diện người dùng. Sự kiện sẽ thông báo cho một phần của chương trình là có điều gì đó đã xảy ra. Tất cả ứng dụng Android đều có một vòng lặp cập nhật giao diện người dùng cốt lõi như bên dưới:



Hình 9.1: Quản lý trạng thái

- Sự kiện: Một sự kiện do người dùng hoặc một phần khác của chương trình tạo ra.
- Trạng thái cập nhật: Trình xử lý sự kiện thay đổi trạng thái mà giao diện người dùng sử dụng.
- Trạng thái hiển thị: Giao diện người dùng được cập nhật để hiển thị trạng thái mới.

Quản lý trạng thái trong Compose nghĩa là hiểu được cách các trạng thái và sự kiện tương tác với nhau. Compose sử dụng hàm remember và mutableStateOf để lưu trữ trạng thái có thể thay đổi trong suốt vòng đời của một Composable.

Ví dụ, tạo một nút để tăng giá trị đếm:

```
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
@Composable
fun Counter() {
    val count = remember { mutableStateOf(0) }
    Button(onClick = { count.value++ }) {
        Text(text = "Đã nhấn: ${count.value} lần")
    }
}
```

Đoạn mã trên đây dùng để mô tả cách quản lý trạng thái với sự kiện xảy ra trên Button:

- @Composable fun Counter(): Đây là một hàm được đánh dấu với từ khóa @Composable, có nghĩa là hàm này sẽ được sử dụng để xây dựng giao diện người dùng (UI) trong Jetpack Compose.
- val count = remember { mutableStateOf(0) }: *remember {}*là hàm lưu trữ giá trị trong bộ nhớ khi thành phần (Composable) được tái cấu trúc (recomposed). Nó đảm bảo rằng biến count sẽ không bị khởi tạo lại mỗi lần giao diện người dùng thay đổi mà vẫn giữ được giá trị của nó giữa các lần vẽ lại (recomposition). *mutableStateOf(0)* tạo một đối tượng trạng thái ban đầu có giá trị là 0. mutableStateOf tạo một trạng thái có thể thay đổi được. Khi giá trị của count thay đổi, Jetpack Compose sẽ tự động vẽ lại thành phần giao diện phụ thuộc vào trạng thái này.
- Giá trị của count: Được lưu trữ trong count.value. Giá trị này sẽ thay đổi khi người dùng nhấn nút và Compose sẽ tự động cập nhật giao diện để hiển thị số lần nhấn mới.
- Button(onClick = { count.value++ }): *Button* là thành phần này tạo ra một nút bấm. Khi nút được nhấn, hành động bên trong onClick sẽ được thực thi. *onClick = { count.value++ }* mỗi lần người dùng nhấn nút, giá trị của count.value sẽ được tăng lên 1. *count.value* là cách để truy cập giá trị hiện tại của đối tượng mutableStateOf.
- Text(text = "Đã nhấn: ${count.value} lần"): *Text* thành phần này hiển thị một chuỗi văn bản lên giao diện. *"Đã nhấn: ${count.value} lần"* là chuỗi hiển thị số lần nút đã được nhấn, với giá trị count.value thay thế vào trong chuỗi thông qua cú pháp nội suy chuỗi ${}.

Cách hoạt động:

- Khi hàm Counter được gọi, count sẽ bắt đầu với giá trị 0.
- Mỗi lần người dùng nhấn nút, hành động trong onClick sẽ tăng giá trị của count.value lên 1.
- Khi count.value thay đổi, Compose tự động vẽ lại giao diện, và văn bản trong thành phần Text sẽ cập nhật số lần nhấn hiện tại.

Ví dụ quản lý trạng thái với EditText:

```
@Composable
fun TextFieldWithStateExample() {
    // Tạo trạng thái để lưu trữ văn bản được nhập vào
    var text by remember { mutableStateOf("") }

    // Bố cục cột để sắp xếp TextField và Text theo chiều dọc
    Column {
        // TextField để người dùng nhập liệu
        TextField(
```

```
            value = text,
            onValueChange = { newText -> text = newText },
            label = { Text("Nhập văn bản của bạn") }
        )

        // Hiển thị văn bản đã nhập bên dưới TextField
        Text(text = "Bạn đã nhập: $text")
    }
}
```

Với khung hiển thị thì người dùng có thể dễ dàng tạo ra EditText và tương tác với nó, tuy nhiên với Compose, để có thể nhập thông tin lên TextFiled và người dùng nhìn thấy được các thông tin này trên giao diện thì cần phải quản lý được trạng thái, do giao diện chỉ cập nhập khi trạng thái thay đổi. Trong ví dụ trên:

- var text by remember { mutableStateOf("") }: Đây là cách chúng ta tạo một trạng thái (state) cho biến text, ban đầu có giá trị là chuỗi rỗng. Trạng thái này sẽ thay đổi khi người dùng nhập văn bản vào.
- TextField: Thành phần cho phép người dùng nhập văn bản. Nó có thuộc tính value để xác định văn bản hiện tại và onValueChange để cập nhật trạng thái mỗi khi người dùng thay đổi nội dung.
- Text: Dùng để hiển thị văn bản mà người dùng đã nhập vào TextField.

Khi người dùng nhập văn bản, trạng thái của text sẽ được cập nhật, và Compose sẽ tự động vẽ lại UI, hiển thị nội dung mới trong Text.

9.1.6. Tích hợp LiveData và ViewModel

Compose dễ dàng tích hợp với LiveData và ViewModel để quản lý dữ liệu trong các ứng dụng thực tế. Để sử dụng ViewModel trong Compose, có thể sử dụng viewModel() để lấy đối tượng ViewModel.

Ví dụ tích hợp ViewModel với LiveData:

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewmodel.compose.viewModel
class MyViewModel : ViewModel() {
    val text = MutableLiveData("Hello from ViewModel")
}
@Composable
fun MyScreen(viewModel: MyViewModel = viewModel()) {
    val text by viewModel.text.observeAsState("")

    Text(text = text)
}
```

Đoạn mã này sử dụng ViewModel để quản lý dữ liệu trong Jetpack Compose. Dữ liệu (ở đây là một chuỗi "Hello from ViewModel") được lưu trữ trong ViewModel thông qua MutableLiveData.

- MyViewModel: Lớp này quản lý dữ liệu, giữ trạng thái của chuỗi text.
- MyScreen: Hàm này quan sát (observeAsState) giá trị của text từ ViewModel. Khi text thay đổi, UI sẽ tự động cập nhật để hiển thị giá trị mới.

Văn bản "Hello from ViewModel" sẽ hiển thị trên màn hình và cập nhật nếu giá trị trong ViewModel thay đổi.

9.1.7. Chủ đề và kiểu dáng

Compose hỗ trợ quản lý theme và styling thông qua các chủ đề tùy chỉnh. Với Material Design, có thể dễ dàng sử dụng các thành phần được định sẵn và áp dụng các chủ đề thông qua MaterialTheme.

```
import androidx.compose.material.MaterialTheme
@Composable
fun ThemedText() {
    MaterialTheme {
        Text(text = "Text theo chủ đề")
    }
}
```

9.1.8. Navigation trong Compose

Navigation là thành phần quan trọng trong ứng dụng Android. Compose cung cấp thư viện navigation-compose để điều hướng giữa các màn hình.

Thêm phụ thuộc:

```
implementation "androidx.navigation:navigation-compose:2.4.0-alpha02"
```

Ví dụ về điều hướng:

```
import androidx.navigation.compose.rememberNavController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
@Composable
fun MyApp() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = "screen1") {
        composable("screen1") { Screen1(navController) }
        composable("screen2") { Screen2() }
    }
}
@Composable
fun Screen1(navController: NavController) {
    Button(onClick = { navController.navigate("screen2") }) {
        Text("Đi tới Screen 2")
    }
}
@Composable
fun Screen2() {
    Text("Đây là Screen 2")
}
```

Ví dụ tạo giao diện như sau bằng Compose

Hình 9.2: giao diện cupcake

```
     @Composable
fun Cake(){
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.SpaceBetween
    ) {
        Column(
            modifier = Modifier.fillMaxWidth(),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {//khoi 1
            //anh
            Spacer(modifier = Modifier.height(50.dp))
            Image(
                modifier = Modifier.size(150.dp),
                painter = painterResource(id = R.drawable.cake6),
                contentDescription = "")
            //text
            Spacer(modifier = Modifier.height(30.dp))
            Text(
                text = "Order Cupcakes",
                fontSize = 30.sp
                )
        }
        Column(
            modifier = Modifier.fillMaxWidth(),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {//khoi 2
            Button(
                onClick = {  },
                modifier = Modifier.fillMaxWidth(),
                colors = ButtonColors(
                    containerColor = Color(0xFFC50C4B),
                    contentColor = Color.White,
```

```
                disabledContainerColor = Color.Gray,
                disabledContentColor = Color.White
            )
            ) {
            Text(text = "Order Now")
        }
        Button(
            onClick = {  },
            modifier = Modifier.fillMaxWidth(),
            colors = ButtonColors(
                containerColor = Color(0xFFC50C4B),
                contentColor = Color.White,
                disabledContainerColor = Color.Gray,
                disabledContentColor = Color.White
            )
            ) {
            Text(text = "Order Now")
        }
        Button(
            onClick = {  },
            modifier = Modifier.fillMaxWidth(),
            colors = ButtonColors(
                containerColor = Color(0xFFC50C4B),
                contentColor = Color.White,
                disabledContainerColor = Color.Gray,
                disabledContentColor = Color.White
            )
        ) {
            Text(text = "Order Now")
        }
    }

}
}
```

Đoạn mã này định nghĩa một hàm Compose có tên Cake, sử dụng các thành phần của Jetpack Compose để tạo giao diện người dùng cho một ứng dụng liên quan đến đặt bánh.

- Column chính: Tạo một cột chiếm toàn bộ kích thước màn hình (fillMaxSize()) với khoảng cách bên trong là 16.dp. Các thành phần bên trong được căn giữa theo chiều ngang và sắp xếp theo chiều dọc với khoảng cách giữa chúng.
- Khối 1:
    - Image: Hiển thị hình ảnh của bánh, có kích thước 150 dp. Có một khoảng cách (Spacer) phía trên và dưới để tạo khoảng trống.
    - Text: Hiển thị tiêu đề "Order Cupcakes" với kích thước chữ 30 sp.
- Khối 2:
    - Button: Có ba nút "Order Now", mỗi nút chiếm toàn bộ chiều rộng. Mỗi nút sử dụng màu sắc tùy chỉnh cho nền và văn bản. Nút được định nghĩa để có màu nền đỏ (mã màu 0xFFC50C4B) và màu chữ trắng. Nếu nút bị vô hiệu hóa, màu nền sẽ chuyển sang màu xám.

9.2. Lập trình phản ứng và View

Ngay từ đầu, Jetpack Compose đã được thiết kế để có khả năng tương tác với Khung hiển thị, tức là Compose và hệ thống Khung hiển thị có thể chia sẻ các tài nguyên và hoạt động bổ trợ cho nhau để hiển thị giao diện người dùng. Chức năng này cho phép bạn thêm Compose vào ứng dụng hiện có dựa trên Khung hiển thị. Tức là, Compose và Khung hiển thị có thể cùng tồn tại trong cơ sở mã của bạn cho đến khi toàn bộ ứng dụng hoàn toàn nằm trong Compose.

Để sử dụng Views trong một dự án Compose, cần thực hiện các bước sau:

- Thêm thư viện Compose vào dự án.
- Sử dụng ComposeView để nhúng UI Compose vào layout XML.
- Định nghĩa các Composable functions cho UI Compose.
- Tạo và quản lý trạng thái bằng LiveData hoặc ViewModel, nếu cần thiết.
- Để sử dụng Views trong một dự án Compose, thực hiện theo các bước sau:

1. Thêm thư viện Compose vào dự án

Cập nhật file build.gradle của module để thêm thư viện Compose. Bật tính năng Compose và thêm các phụ thuộc cần thiết.

```
android {
    buildFeatures {
        compose true
    }
    composeOptions {
        kotlinCompilerExtensionVersion = '1.5.0'
    }
}

dependencies {
    implementation "androidx.compose.ui:ui:1.0.0"
    implementation "androidx.activity:activity-compose:1.3.0"
}
```

Sử dụng ComposeView trong Layout XML

Trong layout XML, sử dụng ComposeView để nhúng các Composable vào ứng dụng dựa trên View.

```
<androidx.compose.ui.platform.ComposeView
    android:id="@+id/compose_view"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

3. Định nghĩa Composable Functions

Tạo các hàm Composable trong mã Kotlin của ứng dụng để định nghĩa các phần tử UI sử dụng Compose.

```
import androidx.compose.material.Text
import androidx.compose.runtime.Composable

@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}
```

4. Thiết lập Compose trong Activity

Trong Activity, tìm ComposeView và thiết lập nó bằng cách sử dụng setContent.

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Greeting(name = "Android")
        }
    }
}
```

5. Quản lý trạng thái (Tùy chọn)

Nếu cần quản lý trạng thái, có thể sử dụng ViewModel và LiveData để kết nối giữa Views và Composable.

```
class MyViewModel : ViewModel() {
    val text = MutableLiveData("Hello from ViewModel")
}
@Composable
fun MyScreen(viewModel: MyViewModel = viewModel()) {
    val text by viewModel.text.observeAsState("")

    Text(text = text)
}
```

Dưới đây là một ví dụ về cách Jetpack Compose tương tác với View System trong một ứng dụng Android. Trong ví dụ này, chúng ta sẽ tạo một giao diện sử dụng cả XML (View System) và Jetpack Compose.

Bước 1: Tạo Layout XML: Trước tiên, tạo một layout XML đơn giản trong res/layout/activity_main.xml. Layout này sẽ có một nút và một FrameLayout để chứa nội dung Jetpack Compose.

```
<!-- res/layout/activity_main.xml -->
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/buttonShowCompose"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hiển thị Compose"
        android:layout_centerInParent="true"/>

    <FrameLayout
        android:id="@+id/composeContainer"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_below="@id/buttonShowCompose"
        android:visibility="gone"/>

</RelativeLayout>
```

Bước 2: Cập Nhật MainActivity: Trong MainActivity, thiết lập để hiển thị nội dung Jetpack Compose trong FrameLayout khi nút được nhấn.

```
import android.os.Bundle
import android.view.View
import android.widget.Button
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.ComposeView
import androidx.compose.ui.tooling.preview.Preview

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val buttonShowCompose: Button =
findViewById(R.id.buttonShowCompose)
        val composeContainer: FrameLayout =
findViewById(R.id.composeContainer)

        buttonShowCompose.setOnClickListener {
            // Khi nút được nhấn, hiển thị Jetpack Compose
            composeContainer.visibility = View.VISIBLE
            val composeView = ComposeView(this)
            composeView.setContent {
                MyComposeScreen()
            }
            composeContainer.addView(composeView)
        }
    }
}

@Composable
fun MyComposeScreen() {
    Surface(modifier = Modifier.fillMaxSize()) {
        Text(text = "Xin chào từ Jetpack Compose!", style =
MaterialTheme.typography.h5)
    }
}

@Preview(showBackground = true)
@Composable
fun MyComposeScreenPreview() {
    MyComposeScreen()
}
```

- Layout XML: Layout này chứa một nút và một FrameLayout. Khi nút được nhấn, FrameLayout sẽ hiển thị nội dung Compose.
- MainActivity: Trong onCreate(), chúng ta tìm nút và FrameLayout. Khi nút được nhấn, chúng ta tạo một ComposeView, thiết lập nội dung của nó bằng cách gọi setContent() với một composable (trong trường hợp này là MyComposeScreen) và thêm nó vào FrameLayout.
- MyComposeScreen: Đây là một composable đơn giản hiển thị một văn bản.

Bài tập bổ trợ

Bài tập thực hành cơ bản

1. Tạo một giao diện hiển thị danh sách tên và tuổi người dùng sử dụng các thành phần Text, Row, và Column trong Jetpack Compose.
2. Quản lý trạng thái của một nút đếm số lần nhấn bằng remember và mutableStateOf.
3. Tạo giao diện tương tác với TextField để người dùng nhập liệu và hiển thị lại dữ liệu đã nhập.
4. Tích hợp LiveData với Jetpack Compose để cập nhật giao diện khi dữ liệu thay đổi.
5. Tạo ứng dụng hiển thị danh sách sản phẩm bằng Jetpack Compose, kết hợp điều hướng (Navigation) giữa các màn hình.

Bài mẫu: Hiển thị danh sách sản phẩm

Mục tiêu: Sử dụng Jetpack Compose để hiển thị danh sách sản phẩm từ một nguồn dữ liệu tĩnh.

Bước thực hiện:

Tạo danh sách dữ liệu:

```
data class Product(val name: String, val price: String)
val products = listOf(
    Product("Laptop", "$1200"),
    Product("Smartphone", "$800"),
    Product("Headphones", "$200"),
    Product("Keyboard", "$100")
)
```

Tạo Composable hiển thị từng sản phẩm:

```
@Composable
fun ProductItem(product: Product) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(8.dp)
    ) {
        Text(
            text = product.name,
            modifier = Modifier.weight(1f),
            style = MaterialTheme.typography.h6
        )
        Text(
            text = product.price,
            style = MaterialTheme.typography.body1
        )
    }
}
```

Hiển thị danh sách sản phẩm:

```
@Composable
fun ProductList(products: List<Product>) {
    Column {
        for (product in products) {
```

```
            ProductItem(product)
        }
    }
}
```

Gọi hàm Composable trong Activity/Fragment:

```
@Composable
fun ProductScreen() {
    ProductList(products)
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ProductScreen()
        }
    }
}
```